

Evaluating Reinforcement Learning methods in Grid-World

ELG 5214 Homework 2 Report

University of Ottawa

1 Environment Design

I implemented a 5×5 grid-world environment containing empty cells, walls, and a goal state (Figure 1). The agent starts from a random position and can take one of four actions: up, down, left, or right.

The reward structure is defined as follows:

- -1 for each step (to encourage efficiency)
- $+10$ for reaching the goal

Episodes terminate either when the goal is reached or when a maximum number of steps is exceeded. This setup creates a sparse reward problem, making exploration an important factor in learning.

2 Methods

2.1 Deep Q-Network (DQN)

DQN is a value-based method that learns an approximation of the action-value function $Q(s, a)$. The agent updates its estimates using the Bellman equation:

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a')$$

To stabilize learning, I used experience replay and a target network.

2.2 REINFORCE

REINFORCE is a policy-gradient method that directly updates the policy parameters using sampled returns:

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi(a|s) G_t]$$

Unlike DQN, REINFORCE does not use bootstrapping, which can lead to higher variance in updates.

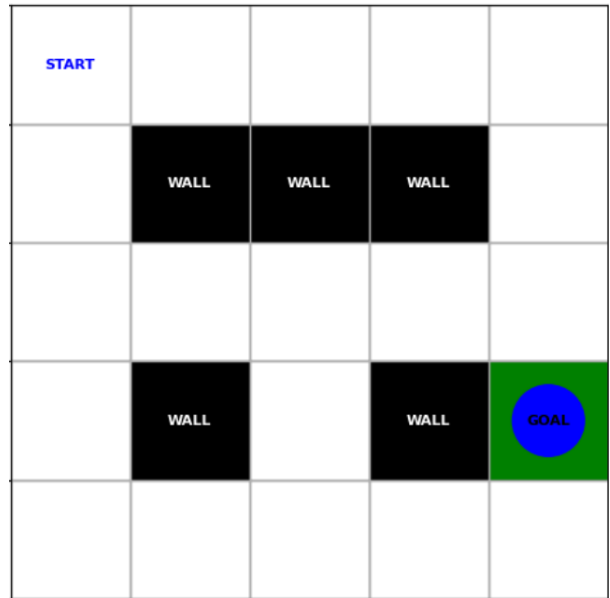


Figure 1: Grid-World Design

2.3 Hyperparameters

This work evaluate the performance of DQN and REINFORCE using a fixed set of hyperparameters and multiple random seeds.

Hyperparameters. A grid search over is performed:

- Learning rates: $\{0.001, 0.01, 0.1\}$
- Discount factors: $\gamma \in \{0.9, 0.99\}$

For DQN, the following parameters are used:

- Batch size: 64
- Target network update frequency: 50 steps
- Replay buffer capacity: 5000
- ϵ -greedy exploration with decay from 1.0 to 0.05

Each configuration is evaluated over 10 independent random seeds. Data recorded:

- Mean return
- Standard error
- Individual seed trajectories

Evaluation Protocol. Each configuration is evaluated over **10 independent random seeds**. For each run, record:

- Episode return

and compute the mean and standard error across seeds.

3 Results

3.1 Learning Curves

The experimental results demonstrate a clear performance gap between DQN, REINFORCE, and the random baseline. As shown in Figure 2 (page 3), DQN exhibits a consistent upward trend in returns over training episodes, eventually achieving positive rewards in several runs. This indicates that the agent successfully learns a policy that reaches the goal efficiently.

In contrast, REINFORCE shows significantly less stable learning behavior. While occasional improvements are observed, the overall performance remains inconsistent and largely negative. The random baseline maintains low and near-constant returns, confirming that the task requires effective learning rather than random exploration.

These results suggest that value-based methods such as DQN are better suited for this grid-world environment, particularly under sparse reward conditions.

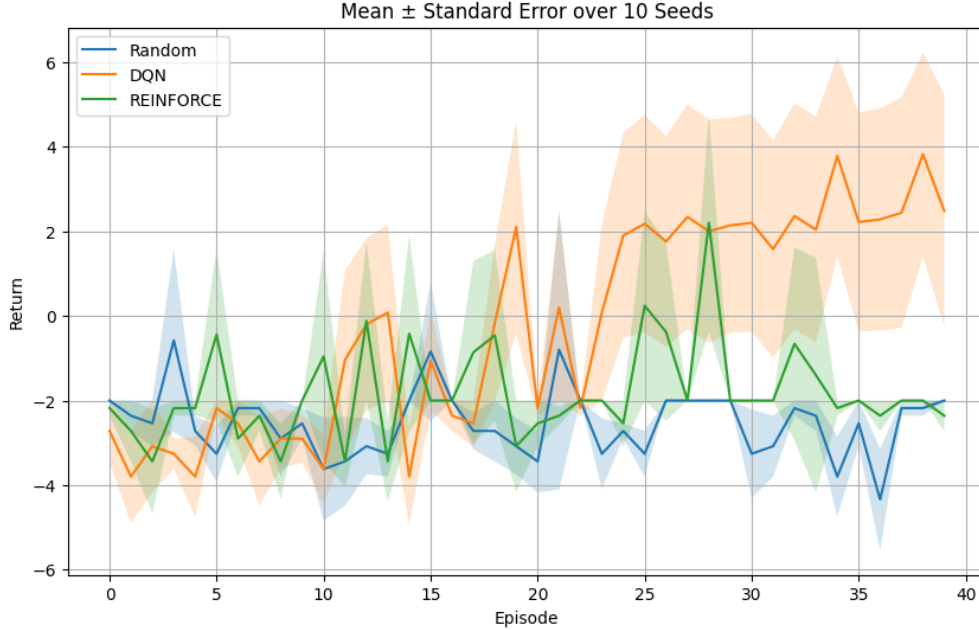


Figure 2: Mean return with standard error over 10 seeds

3.2 Variability Across Seeds

The individual seed trajectories (Figure 3, page 4) reveal substantial variability in performance across different random initializations.

For DQN, several seeds converge to high-performing policies, while others remain suboptimal. This indicates sensitivity to exploration and initialization, but still demonstrates the ability of DQN to reliably discover good solutions.

REINFORCE exhibits significantly higher variance, with large fluctuations and unstable returns across nearly all seeds. This behavior is expected due to the Monte Carlo nature of policy gradient methods, which rely on high-variance return estimates.

Overall, DQN provides more stable and reproducible learning compared to REINFORCE.

3.3 Hyperparameter Sensitivity

Among all configurations, the best performance for DQN was observed at:

$$\text{learning rate} = 0.01, \quad \gamma = 0.9$$

Higher learning rates led to instability, while lower learning rates slowed down learning. A smaller discount factor performed better due to the short horizon of the grid-world task.

3.4 Policy Visualization

The learned policies show directional behavior toward the goal. However, some inconsistencies remain, indicating incomplete convergence.

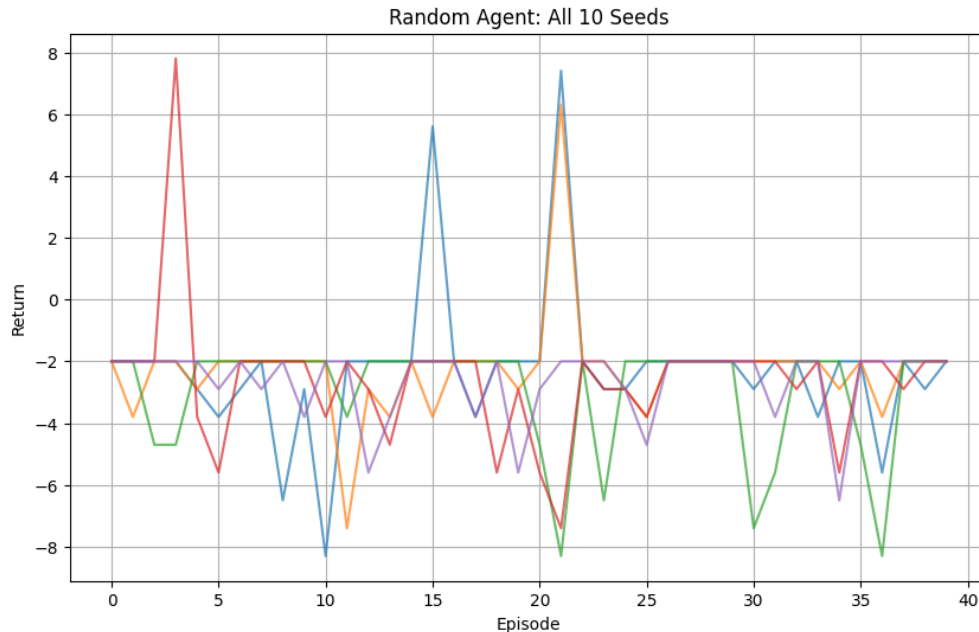


Figure 3: Random Baseline

4 Extension: Symbolic Exploration Bonus (used in our project)

To better connect this assignment with our research project, we introduce a symbolic exploration bonus. The agent receives a small additional reward when visiting a state for the first time within an episode.

This encourages the agent to explore new areas of the environment rather than repeatedly visiting the same states. The modification is simple but effective in promoting broader state coverage.

5 Connection to Final Project

This extension is inspired by our research project on exploration in complex environments such as Pokémon Red. In that setting, traditional approaches rely on pixel-based novelty measures, which can be unstable due to irrelevant visual noise.

Our project proposes using symbolic representations such as (X, Y, MapID) coordinates to guide exploration. The grid-world environment naturally aligns with this idea, as states are already represented in a low-dimensional symbolic form.

The exploration bonus implemented here can be seen as a simplified version of symbolic coordinate discovery. Instead of using high-dimensional visual features, the agent is rewarded for discovering new states in a structured space.

This demonstrates how symbolic exploration can improve learning efficiency and stability, especially in environments with sparse rewards.

6 Discussion

The experiments highlight several important challenges in reinforcement learning:

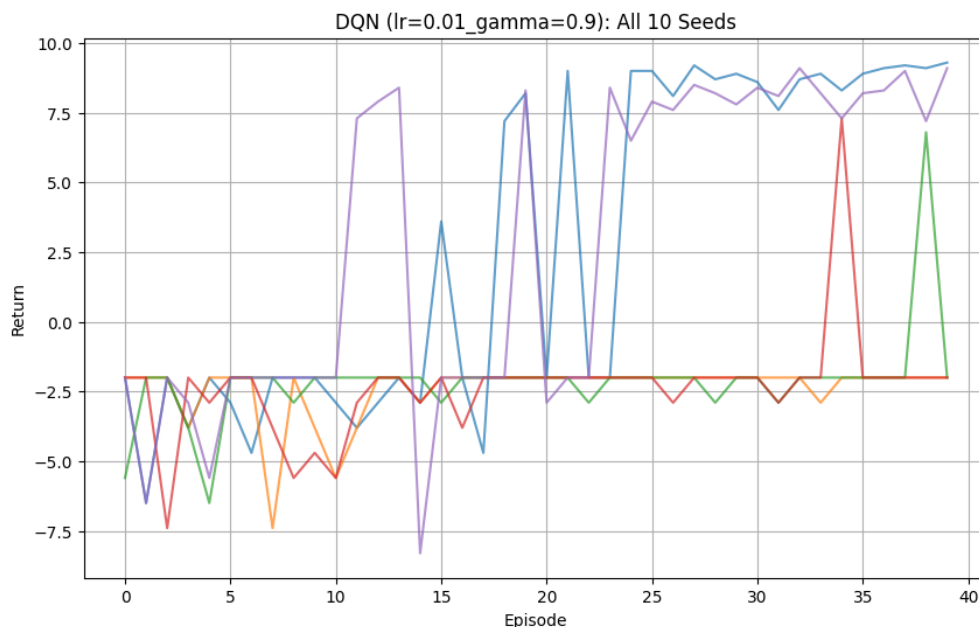


Figure 4: Learned DQN policy

- Sparse rewards make learning difficult
- Exploration plays a critical role in performance
- Variability across seeds can be significant

DQN shows more stable performance due to bootstrapping and replay mechanisms, while REINFORCE suffers from higher variance.

The exploration bonus provides a simple but meaningful improvement by encouraging state discovery, aligning with modern research directions in intrinsic motivation.

7 Implementation Details

All implementations were written using JAX, which enforces a functional programming style. This requires explicit handling of model parameters and randomness but improves reproducibility.

Experiments were conducted on Google Colab using GPU acceleration. However, due to the small scale of the environment, GPU usage did not significantly reduce runtime.

8 GPU usage

Google Colab G4 GPU

9 AI usage

ChatGPT was used on debug, providing suggestions on hyperparameters, and used on providing instruction on adjusting JAX with Colab.

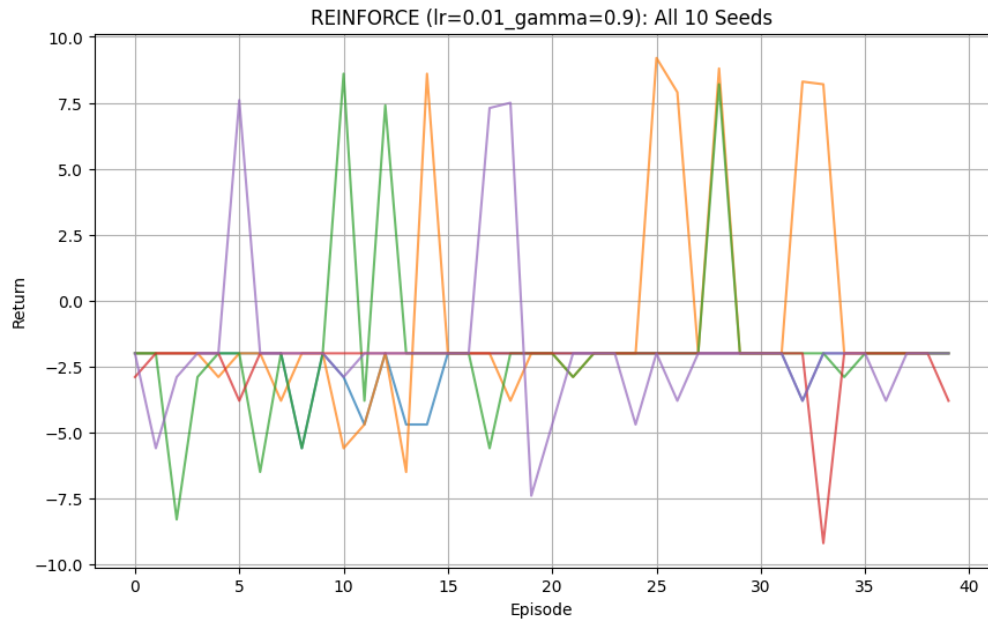


Figure 5: Learned REINFORCE policy

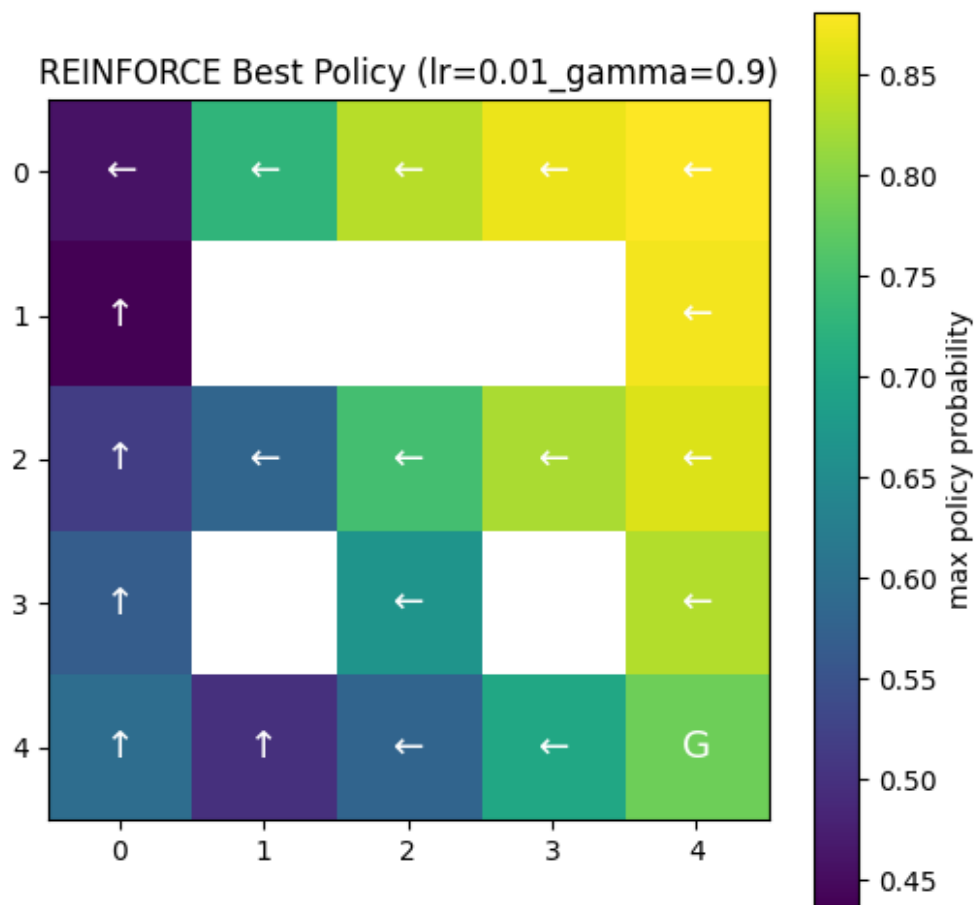


Figure 6: Heatmap Reinforce



Figure 7: Heatmap DQN